

Hypothesis Function:

$$\hat{y} = \theta_0 + \theta_1 x$$

Mean Squared Error (MSE) Cost Function:

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$

Gradient Descent Update Rules:

$$\theta_0 := \theta_0 - \alpha \frac{\partial J}{\partial \theta_0}$$

$$\theta_1 := \theta_1 - \alpha \frac{\partial J}{\partial \theta_1}$$

Gradients:

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})$$

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) x^{(i)}$$

where m = number of training examples and α = learning rate.

Problem

Given a small dataset with three points:

$(1, 2), (2, 3), (3, 5)$

Initialize the parameters at $\theta_0 = 0$ and $\theta_1 = 0$. Use a learning rate of $\alpha = 0.1$. Perform one full iteration of gradient descent to find the new values of θ_0 and θ_1 .

Step-by-Step Application:

1. Calculate Hypotheses:

Using $h_{\theta}(x) = 0 + 0(x)$:

- For $x^{(1)} = 1$: $h_{\theta}(1) = 0$
- For $x^{(2)} = 2$: $h_{\theta}(2) = 0$
- For $x^{(3)} = 3$: $h_{\theta}(3) = 0$

2. Calculate Errors ($h_{\theta}(x^{(i)}) - y^{(i)}$):

- $e_1 = 0 - 2 = -2$
- $e_2 = 0 - 3 = -3$
- $e_3 = 0 - 5 = -5$

3. Compute Gradients:

- Sum of errors: $(-2) + (-3) + (-5) = -10$
- Sum of (error $\times x$): $(-2 \times 1) + (-3 \times 2) + (-5 \times 3) = -2 - 6 - 15 = -23$
- Gradient for θ_0 : $\frac{1}{3}(-10) = -3.333$
- Gradient for θ_1 : $\frac{1}{3}(-23) = -7.667$

4. Update Parameters:

- $\theta_0 = 0 - (0.1 \times -3.333) = 0.333$
- $\theta_1 = 0 - (0.1 \times -7.667) = 0.767$

Result: After one iteration, the model becomes $h_{\theta}(x) = 0.333 + 0.767x$.

Practice Problem 2: Convergence and Cost Calculation

Problem Statement:

Using the updated parameters from Problem 1 ($\theta_0 = 0.333$, $\theta_1 = 0.767$), calculate the initial cost J and the new cost after the first update to verify if the cost function has decreased.

Step-by-Step Application:

1. Initial Cost (at $\theta = 0, 0$):

$$J = \frac{1}{2 \times 3}((-2)^2 + (-3)^2 + (-5)^2) = \frac{1}{6}(4 + 9 + 25) = \frac{38}{6} \approx 6.333$$

2. New Hypotheses:

- $h(1) = 0.333 + 0.767(1) = 1.100$
- $h(2) = 0.333 + 0.767(2) = 1.867$
- $h(3) = 0.333 + 0.767(3) = 2.634$

3. New Errors:

- $e_1 = 1.100 - 2 = -0.900$
- $e_2 = 1.867 - 3 = -1.133$
- $e_3 = 2.634 - 5 = -2.366$

4. New Cost Calculation:

$$J = \frac{1}{6}((-0.900)^2 + (-1.133)^2 + (-2.366)^2) = \frac{1}{6}(0.810 + 1.284 + 5.598) = \frac{7.692}{6} \approx 1.282$$

Result: The cost decreased from 6.333 to 1.282, indicating that the gradient descent step moved the parameters closer to the global minimum.

given the following dataset of house sizes (in hundreds of sq ft) and their prices (in lakhs):

x (Size)	y (Price)
1	2
2	4
3	6

Initialize $\theta_0 = 0$, $\theta_1 = 0$, and learning rate $\alpha = 0.1$. Perform **one iteration** of gradient descent and compute the updated parameters.

Step 1: Define the Hypothesis

$$\hat{y} = \theta_0 + \theta_1 x = 0 + 0 \cdot x = 0$$

Step 2: Compute Predictions and Errors

x	y	$\hat{y}$	Error ($\hat{y} - y$)
1	2	0	-2
2	4	0	-4
3	6	0	-6

Step 3: Compute the Cost Function

$$J(\theta_0, \theta_1) = \frac{1}{2 \times 3} [(-2)^2 + (-4)^2 + (-6)^2]$$

$$J = \frac{1}{6} [4 + 16 + 36] = \frac{56}{6} \approx 9.33$$

Step 4: Compute the Gradients

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{3} [(-2) + (-4) + (-6)] = \frac{-12}{3} = -4$$

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{3} [(-2)(1) + (-4)(2) + (-6)(3)] = \frac{-2 - 8 - 18}{3} = \frac{-28}{3} \approx -9.33$$

Step 5: Update Parameters

$$\theta_0 := 0 - 0.1 \times (-4) = 0 + 0.4 = \boxed{0.4}$$

$$\theta_1 := 0 - 0.1 \times (-9.33) = 0 + 0.933 = \boxed{0.933}$$

Interpretation: After one iteration, the line shifts from $\hat{y} = 0$ to $\hat{y} = 0.4 + 0.933x$, getting closer to the true relationship $y = 2x$.

Given the data set

x	y
1	3
2	5

Initialize $\theta_0 = 1$, $\theta_1 = 0.5$, learning rate $\alpha = 0.1$. Perform **two iterations** of gradient descent. Track the cost at each step.

Iteration 1

Step 1: Compute Predictions

$$\hat{y}^{(1)} = 1 + 0.5 \times 1 = 1.5$$

$$\hat{y}^{(2)} = 1 + 0.5 \times 2 = 2.0$$

Step 2: Compute Errors

$$e^{(1)} = 1.5 - 3 = -1.5$$

$$e^{(2)} = 2.0 - 5 = -3.0$$

Step 3: Compute Cost

$$J = \frac{1}{2 \times 2} [(-1.5)^2 + (-3.0)^2] = \frac{1}{4} [2.25 + 9] = \frac{11.25}{4} = 2.8125$$

Step 4: Compute Gradients

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{2} [-1.5 + (-3.0)] = \frac{-4.5}{2} = -2.25$$

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{2} [(-1.5)(1) + (-3.0)(2)] = \frac{-1.5 - 6}{2} = \frac{-7.5}{2} = -3.75$$

Step 5: Update Parameters

$$\theta_0 := 1 - 0.1 \times (-2.25) = 1 + 0.225 = 1.225$$

$$\theta_1 := 0.5 - 0.1 \times (-3.75) = 0.5 + 0.375 = 0.875$$

Iteration 2

Step 1: Compute Predictions

$$\hat{y}^{(1)} = 1.225 + 0.875 \times 1 = 2.1$$

$$\hat{y}^{(2)} = 1.225 + 0.875 \times 2 = 2.975$$

Step 2: Compute Errors

$$e^{(1)} = 2.1 - 3 = -0.9$$

$$e^{(2)} = 2.975 - 5 = -2.025$$

Step 3: Compute Cost

$$J = \frac{1}{4} [(-0.9)^2 + (-2.025)^2] = \frac{1}{4} [0.81 + 4.1006] = \frac{4.9106}{4} \approx 1.2277$$

Step 4: Compute Gradients

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{2} [-0.9 + (-2.025)] = \frac{-2.925}{2} = -1.4625$$

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{2} [(-0.9)(1) + (-2.025)(2)] = \frac{-0.9 - 4.05}{2} = \frac{-4.95}{2} = -2.475$$

Step 5: Update Parameters

$$\theta_0 := 1.225 - 0.1 \times (-1.4625) = 1.225 + 0.14625 \approx 1.371$$

$$\theta_1 := 0.875 - 0.1 \times (-2.475) = 0.875 + 0.2475 \approx 1.123$$

Iteration	$J(\theta)$
0 (Initial)	2.8125
1	1.2277
Converging toward	~ 0

Predict exam scores based on **hours studied** (x_1) and **hours slept** (x_2):

x_1	x_2	y
2	8	70
4	7	80
6	6	90

Hypothesis: $\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2$

Initialize all parameters at 0: $\theta_0 = \theta_1 = \theta_2 = 0, \alpha = 0.01$.

Step 1: Compute Predictions (all zero initially)

$$\hat{y}^{(1)} = 0, \quad \hat{y}^{(2)} = 0, \quad \hat{y}^{(3)} = 0$$

Step 2: Compute Errors

$$e^{(1)} = 0 - 70 = -70$$

$$e^{(2)} = 0 - 80 = -80$$

$$e^{(3)} = 0 - 90 = -90$$

Step 3: Compute Cost

$$J = \frac{1}{6} [4900 + 6400 + 8100] = \frac{19400}{6} \approx 3233.33$$

Step 4: Compute Gradients

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{3} [-70 + (-80) + (-90)] = \frac{-240}{3} = -80$$

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{3} [(-70)(2) + (-80)(4) + (-90)(6)] = \frac{-140 - 320 - 540}{3} = \frac{-1000}{3} \approx -333.33$$

$$\frac{\partial J}{\partial \theta_2} = \frac{1}{3} [(-70)(8) + (-80)(7) + (-90)(6)] = \frac{-560 - 560 - 540}{3} = \frac{-1660}{3} \approx -553.33$$

Step 5: Update Parameters

$$\theta_0 := 0 - 0.01 \times (-80) = 0.8$$

$$\theta_1 := 0 - 0.01 \times (-333.33) \approx 3.333$$

$$\theta_2 := 0 - 0.01 \times (-553.33) \approx 5.533$$

New hypothesis after 1 iteration:

$$\hat{y} = 0.8 + 3.333x_1 + 5.533x_2$$

given the following dataset of house sizes (in hundreds of sq ft) and their prices (in lakhs):

x (Size)	y (Price)
1	2
2	4
3	6

Compare what happens with:

- $\alpha = 0.01$ (small)
- $\alpha = 0.1$ (moderate)
- $\alpha = 1.0$ (too large)

After **one gradient step**, starting from $\theta_0 = 0, \theta_1 = 0$:

Gradients are computed: $\frac{\partial J}{\partial \theta_0} = -4, \quad \frac{\partial J}{\partial \theta_1} = -9.33$

Learning Rate α	θ_0	θ_1	Behavior
0.01	0.04	0.0933	Slow convergence
0.1	0.4	0.933	Good convergence
1.0	4.0	9.33	Likely divergence (overshooting)

Key Insight: A learning rate that is too large causes J to increase instead of decrease, a phenomenon called **overshooting**. A learning rate too small makes training impractically slow.

1. Basic Linear Regression Setup

For linear regression,

$$\hat{y} = X\theta$$

where

$$\theta = \begin{bmatrix} b \\ w \end{bmatrix}$$

and

$$X = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_m \end{bmatrix}$$

The cost function is

$$J(\theta) = \frac{1}{2m} (X\theta - y)^T (X\theta - y)$$

and the gradient is

$$\nabla J(\theta) = \frac{1}{m} X^T (X\theta - y)$$

The parameter update is

$$\theta_{\text{new}} = \theta - \alpha \nabla J(\theta)$$

The difference between the three methods is **how many training examples are used to calculate each update**:

Method	Examples per update
Batch GD	All training examples
SGD	One training example
Mini-Batch GD	A subset of training examples

Problem 1 — Batch Gradient Descent

Consider the following dataset:

(x)	(y)
1	2
2	4
3	6
4	8

Use:

$$\hat{y} = b + wx$$

Initial parameters:

$$b = 0, \quad w = 0$$

Learning rate:

$$\alpha = 0.1$$

Apply **one Batch Gradient Descent update**.

Step 2: Calculate predictions

$$X\theta = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Step 3: Calculate error

$$e = X\theta - y$$

$$e = \begin{bmatrix} -2 \\ -4 \\ -6 \\ -8 \end{bmatrix}$$

Step 4: Calculate gradient

$$\nabla J = \frac{1}{4} X^T e$$

$$X^T = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \end{bmatrix}$$

Therefore,

$$X^T e = \begin{bmatrix} -20 \\ -60 \end{bmatrix}$$

Hence:

$$\nabla J = \begin{bmatrix} -5 \\ -15 \end{bmatrix}$$

Step 5: Update

$$\theta_{\text{new}} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.1 \begin{bmatrix} -5 \\ -15 \end{bmatrix}$$

$$\theta_{\text{new}} = \begin{bmatrix} 0.5 \\ 1.5 \end{bmatrix}$$

Therefore:

$$\hat{y} = 0.5 + 1.5x$$

Problem 2 — Stochastic Gradient Descent

Initially:

$$b = 0, \quad w = 0$$

and

$$\alpha = 0.1.$$

Apply **SGD** using the first training example only:

$$(x, y) = (1, 2)$$

For one example:

$$\hat{y} = b + wx$$

Initially:

$$\hat{y} = 0$$

Error:

$$e = \hat{y} - y = 0 - 2 = -2$$

For one sample:

$$\frac{\partial J}{\partial b} = e$$

$$\frac{\partial J}{\partial w} = ex$$

Therefore:

$$\frac{\partial J}{\partial b} = -2$$

$$\frac{\partial J}{\partial w} = (-2)(1) = -2$$

Update:

$$b = 0 - 0.1(-2) = \boxed{0.2}$$

$$w = 0 - 0.1(-2) = \boxed{0.2}$$

Thus:

$$\hat{y} = 0.2 + 0.2x$$

Important observation

Batch GD used **all four examples** before making one update.

SGD used **only one example** to make the update.

Problem 3 — Mini-Batch Gradient Descent: Divide the dataset into two mini-batches:

Mini-batch 1

$$(1, 2), (2, 4)$$

Mini-batch 2

$$(3, 6), (4, 8)$$

Initially:

$$b = 0, \quad w = 0$$

and

$$\alpha = 0.1.$$

Apply one update using **Mini-Batch 1**.

For Mini-Batch 1:

$$X_B = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}$$

$$y_B = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$$

Initial:

$$\theta = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Prediction:

$$X_B \theta = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Error:

$$e = \begin{bmatrix} -2 \\ -4 \end{bmatrix}$$

Gradient:

$$\nabla J_B = \frac{1}{2} X_B^T e$$

$$X_B^T = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}$$

Therefore:

$$X_B^T e = \begin{bmatrix} -6 \\ -10 \end{bmatrix}$$

Thus:

$$\nabla J_B = \begin{bmatrix} -3 \\ -5 \end{bmatrix}$$

Update:

$$\theta_{\text{new}} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.1 \begin{bmatrix} -3 \\ -5 \end{bmatrix}$$

$$\theta_{\text{new}} = \begin{bmatrix} 0.3 \\ 0.5 \end{bmatrix}$$

Therefore:

$$\hat{y} = 0.3 + 0.5x$$

Method	Samples used	(b)	(w)
Batch GD	4	0.5	1.5
Mini-Batch GD	2	0.3	0.5
SGD	1	0.2	0.2

Evaluation Metrics

Calculate MAE

Actual and predicted values are:

Actual (y)	Predicted ($\hat{y}$)
10	12
20	18
30	29
40	43
50	48

Calculate **Mean Absolute Error (MAE)**.

Solution

Formula:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

(y)	($\hat{y}$)	Error	Absolute Error
10	12	-2	2
20	18	2	2
30	29	1	1
40	43	-3	3
50	48	2	2

Therefore:

$$MAE = \frac{2 + 2 + 1 + 3 + 2}{5}$$

$$MAE = 2$$

Interpretation: On average, predictions differ from actual values by **2 units**.

Using the same data, calculate **Mean Squared Error**.

Solution

$$MSE = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

Squared errors:

4, 4, 1, 9, 4

Therefore:

$$MSE = \frac{4 + 4 + 1 + 9 + 4}{5}$$

$$MSE = 4.4$$

Calculating RMSE

Using the previous problem:

$$MSE = 4.4$$

Calculate RMSE.

Solution

$$RMSE = \sqrt{MSE}$$

Therefore:

$$RMSE = \sqrt{4.4}$$

$$RMSE \approx 2.098$$

Thus, the typical prediction error is approximately **2.10 units**.

Consider the following data

Actual Model A Model B

10	11	10
20	21	19
30	29	30
40	41	45
50	49	50

Which model is better according to MAE, MSE and RMSE?

For Model A:

Absolute errors:

1, 1, 1, 1, 1

Therefore:

$$MAE_A = 1$$

$$MSE_A = 1$$

$$RMSE_A = 1$$

For Model B:

Absolute errors:

0, 1, 0, 5, 0

Therefore:

$$MAE_B = \frac{6}{5} = 1.2$$

Squared errors:

0, 1, 0, 25, 0

$$MSE_B = \frac{26}{5} = 5.2$$

$$RMSE_B = \sqrt{5.2} \approx 2.28$$

Metric Model A Model B Better

MAE 1.0 1.2 A

MSE 1.0 5.2 A

RMSE 1.0 2.28 A

Model A is clearly better because Model B has one large error, which is heavily penalized by MSE and RMSE.

Calculate R^2

Actual (y) Predicted ($\hat{y}$)

2	2.5
4	3.5
6	5.5
8	8.5
10	9.5

The formula is:

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

where:

$$SS_{res} = \sum (y_i - \hat{y}_i)^2$$

and

$$SS_{tot} = \sum (y_i - \bar{y})^2$$

First:

$$\bar{y} = \frac{2 + 4 + 6 + 8 + 10}{5} = 6$$

Calculate residual sum of squares:

$$SS_{res} = (-0.5)^2 + (0.5)^2 + (0.5)^2 + (-0.5)^2 + (0.5)^2$$
$$SS_{res} = 1.25$$

Total sum of squares:

$$SS_{tot} = (2 - 6)^2 + (4 - 6)^2 + (6 - 6)^2 + (8 - 6)^2 + (10 - 6)^2$$
$$= 16 + 4 + 0 + 4 + 16 = 40$$

Therefore:

$$R^2 = 1 - \frac{1.25}{40}$$

$$R^2 = 0.96875$$

or approximately:

$$R^2 = 96.88\%$$

Interpretation: Approximately **96.88%** of the variance in the dependent variable is explained by the regression model.

Min-Max Normalization

Given the feature:

$$X = [10, 20, 30, 40, 50]$$

Normalize the values to the range $[0, 1]$.

Solution

Min-Max normalization:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Here:

$$x_{\min} = 10, \quad x_{\max} = 50$$

Original	Normalized
10	0
20	0.25
30	0.50
40	0.75
50	1.00

Standardization

Given:

$$X = [10, 20, 30, 40, 50]$$

Calculate the standardized values.

Solution

Z-score normalization:

$$z = \frac{x - \mu}{\sigma}$$

Mean:

$$\mu = \frac{10 + 20 + 30 + 40 + 50}{5}$$

$$\mu = 30$$

Population standard deviation:

$$\sigma = \sqrt{\frac{(10 - 30)^2 + (20 - 30)^2 + (30 - 30)^2 + (40 - 30)^2 + (50 - 30)^2}{5}}$$

$$= \sqrt{\frac{400 + 100 + 0 + 100 + 400}{5}}$$

$$= \sqrt{200}$$

$$\sigma = 14.142$$

Now:

$$z_{10} = \frac{10 - 30}{14.142} = -1.414$$

$$z_{20} = \frac{20 - 30}{14.142} = -0.707$$

$$z_{30} = 0$$

$$z_{40} = 0.707$$

$$z_{50} = 1.414$$

Answer

$$[-1.414, -0.707, 0, 0.707, 1.414]$$

Robust Scaling

Consider:

$$X = [10, 20, 30, 40, 1000]$$

Use **Robust Scaling**.

Formula:

$$x' = \frac{x - \text{Median}}{IQR}$$

Solution

Sorted data:

$$10, 20, 30, 40, 1000$$

Median:

$$\text{Median} = 30$$

Lower half:

$$10, 20$$

$$Q_1 = 15$$

Upper half:

$$40, 1000$$

$$Q_3 = 520$$

Therefore:

$$\begin{aligned} IQR &= Q_3 - Q_1 \\ &= 520 - 15 = 505 \end{aligned}$$

Now:

$$x' = \frac{x - 30}{505}$$

(x) Robust-scaled

10	-0.0396
20	-0.0198
30	0
40	0.0198
1000	1.9208

Robust scaling uses:

Median and IQR

rather than mean and standard deviation, making it generally more resistant to outliers.

Summary Formula Sheet

1. Min-Max

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

For arbitrary range $[a, b]$:

$$x' = a + \frac{(x - x_{\min})(b - a)}{x_{\max} - x_{\min}}$$

2. Z-Score Standardization

$$z = \frac{x - \mu}{\sigma}$$

3. Mean Normalization

$$x' = \frac{x - \mu}{x_{\max} - x_{\min}}$$

4. Decimal Scaling

$$x' = \frac{x}{10^j}$$

5. L1 Normalization

$$x'_i = \frac{x_i}{\sum_j |x_j|}$$

6. L2 Normalization

$$x'_i = \frac{x_i}{\sqrt{\sum_j x_j^2}}$$

7. Robust Scaling

$$x' = \frac{x - \text{Median}}{IQR}$$

where

$$IQR = Q_3 - Q_1$$

1. Basic L2 Regularization — Ridge Regression

Consider the linear regression model:

$$\hat{y} = w_0 + w_1 x$$

The training data is:

(x)	(y)
1	2
2	4
3	6

Suppose:

$$w_0 = 0, \quad w_1 = 2$$

and

$$\lambda = 0.1$$

Calculate the **regularized cost** using L2 regularization.

Use:

$$J(w) = \frac{1}{2m} \sum_{i=1}^m (y_i - \hat{y}_i)^2 + \frac{\lambda}{2} \sum_{j=1}^n w_j^2$$

Do not regularize w_0 .

Solution

Predictions:

$$\hat{y} = [2, 4, 6]$$

Therefore:

$$y - \hat{y} = [0, 0, 0]$$

Hence:

$$MSE = 0$$

Regularization penalty:

$$\begin{aligned} & \frac{\lambda}{2} w_1^2 \\ &= \frac{0.1}{2} (2)^2 \end{aligned}$$

$$= 0.05(4) = 0.2$$

Therefore:

$$J(w) = 0.2$$

Although the training error is zero, the regularized cost is not zero because the model has a non-zero weight.

2. Calculate L2 Penalty for Multiple Features

Consider:

$$w = \begin{bmatrix} 2 \\ -3 \\ 4 \end{bmatrix}$$

and:

$$\lambda = 0.2$$

Calculate the L2 regularization penalty.

L2 penalty:

$$P_{L2} = \frac{\lambda}{2} \sum_j w_j^2$$

Therefore:

$$\begin{aligned} P_{L2} &= \frac{0.2}{2} (2^2 + (-3)^2 + 4^2) \\ &= 0.1(4 + 9 + 16) \\ &= 0.1(29) \end{aligned}$$

$$\boxed{P_{L2} = 2.9}$$

3. Calculate L1 Penalty

Given:

$$w = \begin{bmatrix} 2 \\ -3 \\ 4 \end{bmatrix}$$

and:

$$\lambda = 0.2$$

Calculate the L1 penalty.

Solution

L1 penalty:

$$P_{L1} = \lambda \sum_j |w_j|$$

Therefore:

$$\begin{aligned} P_{L1} &= 0.2(|2| + |-3| + |4|) \\ &= 0.2(2 + 3 + 4) \end{aligned}$$

$$\boxed{P_{L1} = 1.8}$$

Ridge Regression Parameter Update

Consider one parameter:

$$w = 5$$

The gradient of the ordinary loss is:

$$\frac{\partial J}{\partial w} = -4$$

Given:

$$\lambda = 0.2$$

and learning rate:

$$\alpha = 0.1$$

Calculate the new parameter using **L2 regularization**.

For L2 regularization:

$$\frac{\partial J_{reg}}{\partial w} = \frac{\partial J}{\partial w} + \lambda w$$

Therefore:

$$= -4 + (0.2)(5)$$

$$= -4 + 1 = -3$$

Gradient descent:

$$w_{new} = w - \alpha \frac{\partial J_{reg}}{\partial w}$$

$$= 5 - 0.1(-3)$$

$$\boxed{w_{new} = 5.3}$$

Consider the following final coefficients after regularization:

Model A — Ridge

$$w = [0.02, 1.8, 0.03, 2.1]$$

Model B — Lasso

$$w = [0, 1.7, 0, 2.0]$$

Which model performs automatic feature selection?

Solution

L1 regularization has a tendency to drive some coefficients exactly to zero:

$$L1 \rightarrow \text{sparse coefficients}$$

Here, Model B has:

$$w_1 = 0, \quad w_3 = 0$$

Therefore features 1 and 3 can effectively be removed.

Ridge generally shrinks coefficients but typically does not make them exactly zero.

Hence:

Model B (Lasso) performs feature selection.

Elastic Net combines L1 and L2 regularization.

The objective function can be written as:

$$J(w) = MSE + \lambda \left[\alpha \sum |w_j| + \frac{1 - \alpha}{2} \sum w_j^2 \right]$$

Suppose:

$$w = [2, -3]$$

$$\lambda = 0.2$$

and:

$$\alpha = 0.5$$

Calculate the regularization penalty.

Solution

L1 component:

$$\sum |w_j| = 2 + 3 = 5$$

L2 component:

$$\begin{aligned} \frac{1}{2} \sum w_j^2 &= \frac{1}{2}(4 + 9) \\ &= 6.5 \end{aligned}$$

Elastic Net penalty:

$$\begin{aligned} &0.2 [0.5(5) + 0.5(6.5)] \\ &= 0.2(2.5 + 3.25) \\ &= 0.2(5.75) \end{aligned}$$

$$\boxed{1.15}$$

Choosing an Appropriate regularization

A dataset contains:

- 500 features
- only 100 training examples
- many irrelevant features
- highly correlated features

Which regularization technique would you consider?

Solution

There is a high risk of:

Overfitting

L1 can perform feature selection, but with highly correlated features it may select one feature and discard another correlated feature somewhat arbitrarily.

Ridge handles multicollinearity well because it shrinks correlated coefficients.

Elastic Net combines both behaviors.

Therefore, a strong candidate is:

Elastic Net

because it provides:

- L1 sparsity
- L2 shrinkage
- improved behavior with correlated predictors

Bias-Variance Trade-Off

Three models have the following results:

Model	Training Error	Validation Error
No Regularization	1	20
Moderate Regularization	5	8
Strong Regularization	15	17

Identify the underfitting/overfitting behavior.

Solution

No Regularization

$$Training = 1$$

$$Validation = 20$$

Very low training error but high validation error.

Overfitting

Moderate Regularization

$$Training = 5$$

$$Validation = 8$$

Good generalization.

Best model

Strong Regularization

$$Training = 15$$

$$Validation = 17$$

Both errors are relatively high.

Underfitting

Therefore:

Moderate regularization provides the best bias-variance trade-off.

Formulas

Ordinary Linear Regression

$$J(\theta) = \frac{1}{2m} \|X\theta - y\|^2$$

Ridge

$$J(\theta) = \frac{1}{2m} \|X\theta - y\|^2 + \frac{\lambda}{2} \sum_{j=1}^p w_j^2$$

Lasso

$$J(\theta) = \frac{1}{2m} \|X\theta - y\|^2 + \lambda \sum_{j=1}^p |w_j|$$

Elastic Net

$$J(\theta) = MSE + \lambda \left[\alpha \sum |w_j| + \frac{1-\alpha}{2} \sum w_j^2 \right]$$

Ridge Normal Equation

$$\theta = (X^T X + \lambda I)^{-1} X^T y$$

with the intercept excluded from the regularization penalty.

Central Concept

Regularization does not primarily aim to reduce training error.

Its purpose is to **control model complexity and improve generalization to unseen data**. Increasing λ generally increases bias while reducing variance; the practical objective is to choose a value that achieves the best validation/test performance.